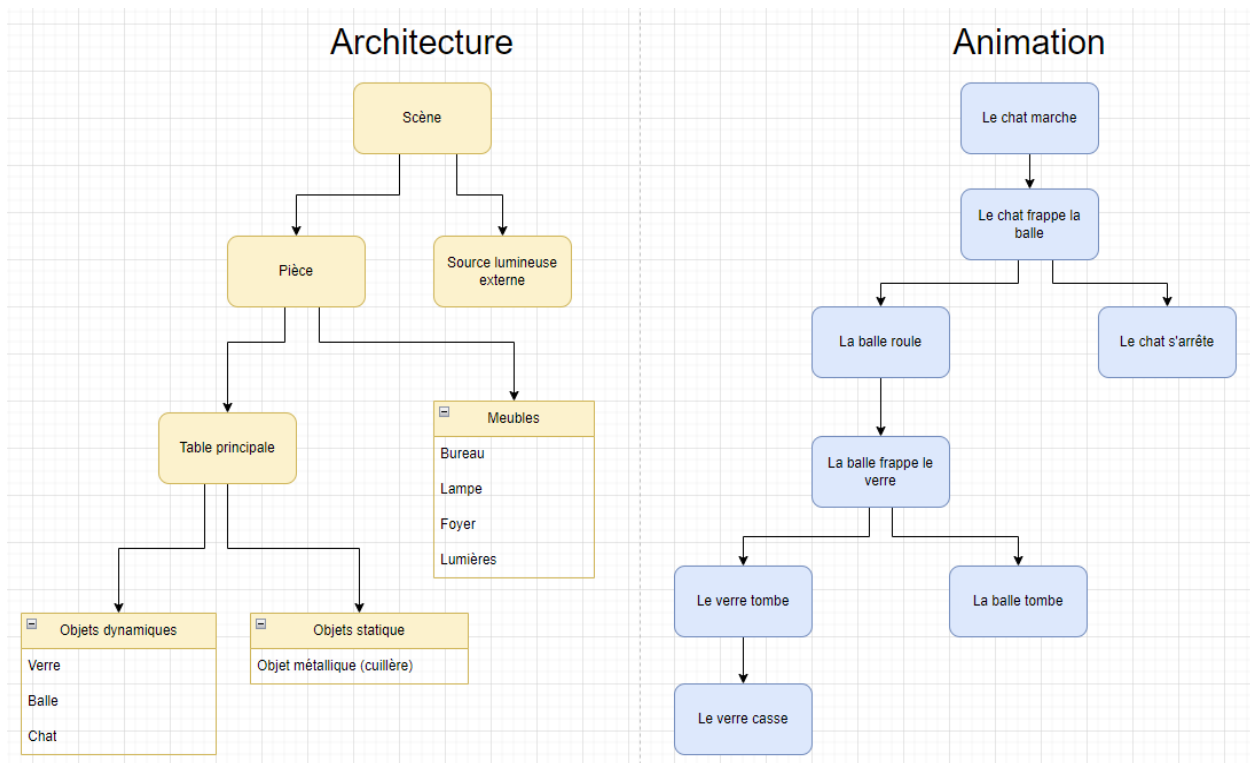


Challenge de rendu et de dynamique physique
INFO4900 - Sujets avancés en informatique
Automne 2025

Dyhia Hamouche
Taieb Jemni
Francis-René McLaughlin
Hugo Sonier

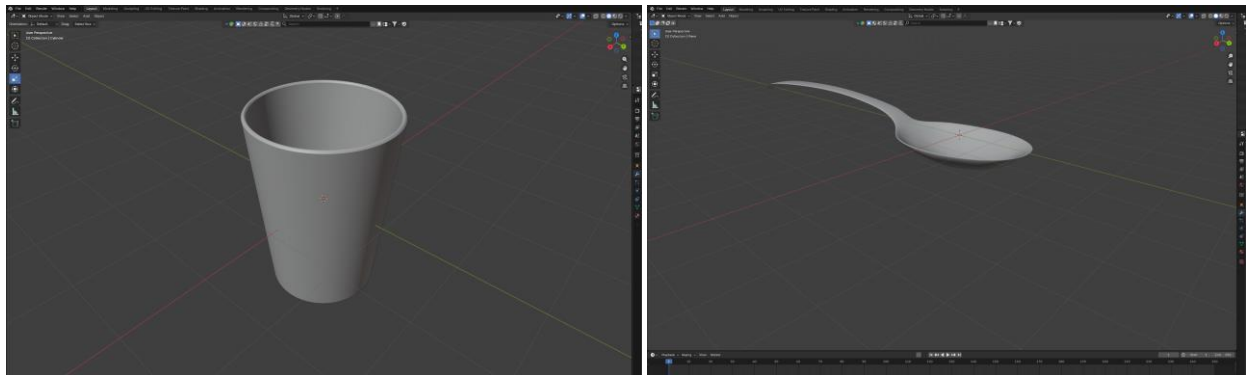
Département d'informatique
Faculté des sciences
Université de Moncton, campus de Moncton
Le 8 décembre 2025

Diagramme bref :



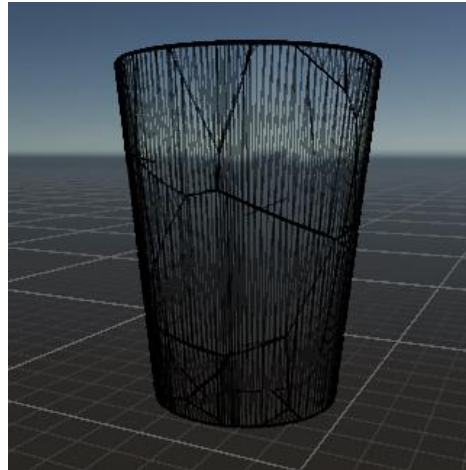
Création du verre et de la cuillère

Nous avons modélisé un verre en utilisant le logiciel Blender. Pour ce faire, un cylindre a été créé et modifié à l'aide des outils de modélisations dans Blender. Nous avons également modélisé une cuillère avec Blender.



Des modificateurs ont été appliqués à ces objets pour obtenir une apparence lisse et augmenter l'épaisseur des objets. Ensuite, ces objets ont été importés dans Unity sous format fbx. Nous avons créé des matériaux dans Unity pour ces deux objets. Un matériel métallique a été configuré pour la cuillère, où la valeur « Metallic » et « Smoothness » ont été assignés au maximum (1.0), afin d'obtenir un effet métallique accentué. Pour le verre, un matériel transparent a été utilisé, avec la valeur « Smoothness » à 0.9 pour obtenir un rendu réaliste. Nous avons également ajouté un script de déformation au verre, afin que celui-ci casse lorsqu'il frappe le sol. Pour ce faire, il a fallu créer

un deuxième modèle pré-cassé dans Blender à l'aide de la fonction « Fracture Cell » et l'importer dans Unity en tant que prefab.

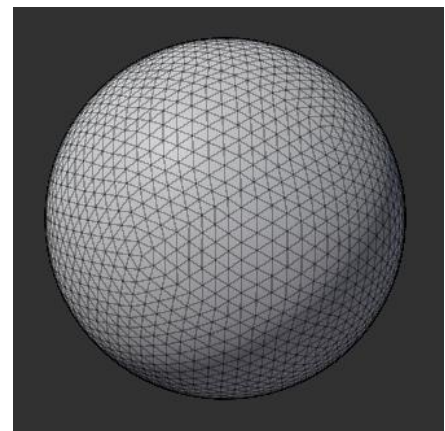


Lorsque le verre touche le sol, le script remplace instantanément le verre intact par le prefab pré-cassé et laisse la physique faire le reste du travail, créant ainsi un effet réaliste.



Création de la balle souple

Pour créer la balle souple, nous avons commencé avec le package ProBuilder dans Unity pour générer une sphère high poly avec 4 subdivisions, puis nous avons appliqué un Rigidbody avec les paramètres « Interpolate » et « Continuous Detection » activés pour essayer de rendre la déformation plus réaliste. Un Physics Material a également été ajouté pour introduire un léger rebond, créant ainsi un comportement réaliste lorsque la balle tombe.



Déformation de la balle

Pour la déformation de la balle, après avoir effectué des recherches, nous avons découvert que Unity ne permet pas nativement de faire des déformations de ce type. Nous avons trouvé deux options principales, soit acheter un asset qui permet la déformation ou créer notre propre script C# personnalisé pour la déformation de la balle. Nous avons opté pour le script C# personnalisé, mais cela a représenté un défi technique important. Les IA génératives ne peuvent pas facilement générer un script de ce type suffisamment réaliste pour que ce soit copier-coller. Il a fallu des heures de tests afin d'obtenir un résultat satisfaisant, malgré l'aide des IA génératives.

Le script capture la position et la force de chaque collision, puis identifie les vertices proches du point d'impact dans un certain rayon. Ces vertices sont déplacés dans la direction opposée à l'impact proportionnellement à leur distance du point de collision. Pour conserver le volume de la balle, le script utilise un ratio de Poisson qui simule le comportement des matériaux élastiques réels. Lorsque la balle est comprimée au point d'impact, les vertices perpendiculaires à la collision sont légèrement poussés vers l'extérieur, créant ainsi une déformation réaliste. La déformation diminue progressivement au fil du temps grâce à une fonction d'interpolation lisse (Lerp), permettant à la balle de retrouver sa forme sphérique originale après chaque impact.



Éclairage

Pour l'éclairage, nous utilisons HDRP (High Definition Render Pipeline). Nous avons créé une fenêtre près de la table à l'aide de géométries Unity et d'un matériau transparent. Une lumière directionnelle (Directional Light) pointe vers cette fenêtre avec des teintes légèrement chaudes pour simuler une lumière du matin. La résolution de la shadowmap a été configurée à 2048 pour assurer une qualité d'ombrage optimale et réduire les bugs.

Un Global Volume a été ajouté pour contrôler finement l'éclairage de la scène. Nous avons configuré une exposition fixe (Fixed Exposure), ajouté un HDRI Sky pour l'éclairage ambiant, ainsi

qu'un brouillard volumétrique (Volumetric Fog) pour rendre visible la lumière dans l'espace 3D et créer des rayons lumineux (god rays) traversant la fenêtre.



Ajout et animation du chat

Nous avons ajouté un chat sous forme 3D provenant de Sketchfab :

- **Titre du modèle :** Realistic Animated Walking Cat 3D Model
- **Auteur :** *burgeonitsolutions*
- **Source :** <https://sketchfab.com/3d-models/realistic-animated-walking-cat-3d-model-92d8b9c4a33b4456ae6e066d85b9a04c>
- **Licence :** Creative Commons Attribution – NonCommercial (CC BY-NC)
--> D'après la licence Le nom de l'auteur est crédité donc l'utilisation est conforme à notre projet académique.

Le modèle a été importé dans Unity en format FBX avec son Armature et la texture. Un Animator Controller a été configuré avec le cycle de marche fourni par le modèle. Pour contrôler le mouvement du chat on a développé un script C# (CatWalk.cs), qui permet au chat de faire une marche vers un point précis (walkPoint). Pour faire cela, il calcule la direction vers la cible et déplace le chat avec MoveTowards et ajuste son orientation vers le point cible (Slerp). Le script fait aussi une interaction physique du chat avec la balle pour cela on a placé Un point (pawHitPoint) sur la patte qui sert à détecter la proximité de la balle et quand la distance devient petite, une force d'impulsion est appliquée sur le rigidbody de la balle et simule un petit coup de patte déclenchant le mouvement de la balle

Nous avons décidé d'ajouter un chat à notre projet pour le rendre plus réaliste. Le chat marche sur la table et pousse la balle avec sa patte, permet d'introduire une dynamique physique cohérente avec le scénario du projet. Donc ce choix apporte une animation et une interaction visuelle supplémentaire et renforce le côté naturel et réel de la scène et ajout un élément narratif qui explique pourquoi la balle commence à rouler

Création de la pièce

Pour construire la pièce dans Unity, nous avons importé plusieurs modèles 3D, prefabs, textures et matériaux provenant de différentes plateformes (Sketchfab, Unity Asset Store, etc.).

Chaque modèle a été importé (souvent en FBX), puis nous avons configuré les matériaux HDRP en appliquant les textures disponibles (Base Map, Normal Map, Mask Map...).

Les objets ont ensuite été placés dans la scène et ajustés pour correspondre au design souhaité.

Sources des assets utilisés

Titre du modèle : Next-gen Furniture Pack

Auteur : Vertex Studio

Source : https://assetstore.unity.com/packages/3d/props/furniture/next-gen-furniture-pack-23303?srltid=AfmBOopyfnDy1XM_ZJLHgZlRtHrI--qCLKH_29ei2FfRW-AKcYOpk-#description

Licence : Standard Unity Asset Store EULA

-> Licence compatible avec un usage académique

Titre du modèle : 4K Tiled Ground Textures (part 2)

Auteur : Awessets

Source : <https://assetstore.unity.com/packages/2d/textures-materials/4k-tiled-ground-textures-part-2-283704#publisher>

Licence : Standard Unity Asset Store EULA

-> Licence compatible avec un usage académique